

```

/*
 * ~Pretend this is in Morty's voice from the iconic tv show rick and
morty
 *
 * oooooo-wee it's finally time for mission #4
 * thi- this code is reeeeeally somethin'
 * th- th- the shell for this code is ummmmm- it's adapted from the
mission #3 code-
 * here's some of the library's you should be aware of:
 *
 *   Adafruit_GPS      -> GPS (Glboal Positioning Systeme))
 *   SoftwareSerial    -> GPS
 *   Wire              -> General
 *   ServoTimer2       -> Servos
 *   Radiohead         -> RF Module
 *
 * Now uh here's how we have uh- our pins/plugins f- f- for the arduino
board
 * it's an uh- arduino mega
 *
 * GPS:
 *   TX  -> 16
 *   RX  -> 17
 *   GND -> GND
 *   PWR -> 5V PWR
 *
 * oh- and I- I- I forgot to mention tha- that this is for uh- team BEVW
 * oooooo-wee I hope you enjoy!
 * oh and please don't tell rick- he would be pretty mad if he found out I
was doing something without him
 *
 * ~End the Morty scene
 */

//Call Libraries
#include <Adafruit_GPS.h>           //GPS
#include <SoftwareSerial.h>         //GPS
#include <ServoTimer2.h>            //Servos
#include <Wire.h>                   //General
#include <RH_ASK.h>                 //Receiver Amplitude Shift Keying (ASK)

```

```

#include <SPI.h>                                //Receiver Serial Peripheral Interface
(SPI)

//GPS Setup
#define GPSSerial Serial2                      //RX 17 AND TX 16
Adafruit_GPS GPS(&GPSSerial);

#define GPSECHO false                          //Disables noise reading from GPS

//General Motor Setup
#define rightMotorspeed 200                    //Speed
#define leftMotorspeed 180                    //Speed
#define steeringLock 70                      //Max Allowable Steering

//Steering Variables
float steer = 0;
float steerGain = 2.0;

//Turning Variables
float msPerDegree = 6.6;
float turnTolerance = 18.0;

//Left Motor Setup (A)
int enA = 3;
int in1 = 4;
int in2 = 5;

//Right Motor Setup (B)
int enB = 8;
int in3 = 6;
int in4 = 7;

//Payload Servo Setup
int servoPayloadPin = 13;
int dropAngle = 1000;
int holdAngle=2000;
ServoTimer2 payloadServo;
bool payloadDropped = false;

//Receiver Setup

```

```

    RH_ASK rf_driver(1000, 11, 0); //Define receiver pin

    int oneTime1 = 0; //variable used to disable the receiver
call function.
    int oneTime2 = 0; //variable used to disable orginal coord
grab function
    int oneTime3 = 0; //variable used to allow for initial
drive state.

//Dynamic Heading Setup
    double targlat = 39.08044; //waypoint/target destination --
*****PLUG IN HARD COORDS
HERE*****
    double targlong = -108.56077;

    double roverlat = 0; //Rover Coords
    double roverlong = 0;

    double lastlat = 0;
    double lastlong = 0;

    double dHead = 0; //Theta of where the rover needs to
point
    double rHead = 0; //Current Theta of rover
    double error = 0; //Error that would need correction

    double origlat; //for grabbing starting coords so
the rover can return to home
    double origlong;

    bool lastposition = false;

//Position to Final Check
    double distance = 0;
    float travel = 0;

//Millis
    uint32_t timer = millis(); //timer for use in the loop
    uint32_t missionStart = millis(); //timer for mission end

```

```

void setup()
{
    Serial.begin(9600);

    //Set all the motor control pins to outputs
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);

    //Turn off motors - Initial state
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);

    //Payload
    payloadServo.attach(servoPayloadPin);
    payloadServo.write(holdAngle);

    //Receiver
    rf_driver.init();

    //GPS
    GPSSerial.begin(9600); //Serial rates
    GPS.begin(9600);

    GPS.sendCommand(PMTK_SET_NMEA_OUTPUT_RMCGGA);
    GPS.sendCommand(PMTK_SET_NMEA_UPDATE_1HZ); // 1 Hz update rate
    GPS.sendCommand(PGCMD_ANTENNA);
}

//=====

void loop()
{

```

```

Stable_GPS();                                //Grabs Position

if(oneTime1 == 0)
    Call_Receiver();                        //Grabs Target Coords ~~
Circumvented if Coords are Hard Coded

if(oneTime2 == 0)
    Original_Point();                      //Stores Starting Point of Rover

if(oneTime3 == 0)
    First_Movement();                    //Provides an Initial Burst of
Starting Movement so Rover can Calibrate its Heading

    Target_Distance();                    //Solves Distance From Target
Location

    if (distance < 3 && payloadDropped)
    {
        Stop();
        payloadServo.write(holdAngle);    //physical flag for debugging
        Serial.end();
        while(true);
    }

    if(distance < 3 && !payloadDropped)    //Replaces
Target Coords & drops payload
    {
        targlat = origlat;
        targlong = origlong;

        Drop_Payload();
        payloadDropped = true;
    }

if(lastposition)
{

```

```

        Dynamic_Heading(); //Calculates Heading Rover Must
Drive from Position

        Travel_Distance();
        if(travel > 2.0)
            Rover_Heading(); //Calculates Heading of Rover
Itself

        else
            rHead = dHead;

        Heading_Error(); //Solves How Much Heading Error
the Rover Must Correct

        Turn(error); //Turn rover corrective amount
to point at final location

        uint32_t driveTime = 8000;
        if(distance < 35)
            driveTime = 3500;

        uint32_t tick = millis() + driveTime; //How long should it drive
for before it looks out again
        while (millis() < tick)
            Drive();

        Stop();
    }

}

//=====

=====

=====

void GPS_Update() //Update GPS
{
    GPS.read();

```

```

    if(GPS.newNMEAreceived())
        !GPS.parse(GPS.lastNMEA());
}

//=====
=====

void Stable_GPS()
{
    //This portion is inspired from team Sandcrawlers code, shouts to them
    <3

    while(true)
    {
        while(!GPS.fix)
            GPS_Update();

        double lat1 = GPS.latitudeDegrees;
        double long1 = GPS.longitudeDegrees;

        uint32_t waitTime = millis();
        while(millis() - waitTime < 2000)
            GPS_Update(); // Keeps the buffer from overflowing!

        while(!GPS.fix)
            GPS_Update();

        double lat2 = GPS.latitudeDegrees;
        double long2 = GPS.longitudeDegrees;

        if(fabs(lat2 - lat1) < 0.00001 && fabs(long2 - long1) < 0.00001)
        //check if the two readings are accurate then take action ~ currently set
        to within 5 ft of eachother
        {

            lastlat = roverlat;    //rover previous points for heading calcs
            lastlong = roverlong;

            roverlat = lat2;        //update current rover position

```

```

        roverlong = long2;

        if(lastlat != 0)
            lastposition = true;

        return;
    }
}

}

//=====
=====
=====

void Call_Receiver()    //circumvented if coords are hard coded.
{
    if(targlong == 0 && targlat == 0)
    {
        uint8_t buf[12];
        uint8_t buflen = sizeof(buf);

        while(oneTime1 == 0)                //Force rover to wait for coords to
be recieved and processed
        {

            if (rf_driver.recv(buf, &buflen)) //Check if received packet is
correct size
            {
                char northStr[6] = {0};
                char westStr[6]  = {0};

                strncpy(northStr, buf + 1, 5);
                strncpy(westStr, buf + 7, 5);

                double northValue = atof(northStr);
                double westValue  = atof(westStr);

```



```

        targlong = (westValue / 100000)*(-1) - 108;
        targlat = (northValue / 100000) + 39;

        oneTime1 ++;
    }
}

else
    oneTime1++;
}

//=====
=====
=====

void Original_Point()
{
    origlat = roverlat;
    origlong = roverlong;

    oneTime2++;           //ensure this only runs once
}

//=====
=====
=====

void First_Movement()
{
    uint32_t tick = millis()+ 8000;
    while (millis() < tick)
    {
        digitalWrite(in1, LOW);    //Turn on motors
        digitalWrite(in2, HIGH);
        digitalWrite(in3, HIGH);
        digitalWrite(in4, LOW);

        analogWrite(enA, leftMotorspeed);
        analogWrite(enB, rightMotorspeed);
    }
}

```

```

    Stop();

    uint32_t tick2 = millis() + 2000;
    while(millis() < tick2)
        GPS_Update();

    Stable_GPS();

    oneTime3++;
}

//=====
=====
=====

void Dynamic_Heading()
//compute adaptable heading error that the rover needs to point so that it
drives to target point
{
    //Please note ChatGPT helped write this portion

    double lat1 = radians(roverlat);        //current position
    double long1 = roverlong;

    double lat2 = radians(targlat);        //target position
    double long2 = targlong;

    double dLon = radians(long2 - long1);

    double x = sin(dLon) * cos(lat2);
    double y = cos(lat1)*sin(lat2) - sin(lat1)*cos(lat2)*cos(dLon);

    dHead = degrees(atan2(x, y));    //solve the dynamic theta

    if (dHead < 0)                    //keep heading error within 360
degrees
        dHead += 360;
}

```

```

//=====
=====

void Rover_Heading()                                     //just
like Dynamic Heading but with different variables
{
    double lat1 = radians(lastlat);                    //current position
    double long1 = lastlong;

    double lat2 = radians(roverlat);                   //target position
    double long2 = roverlong;

    double dLon = radians(long2 - long1);

    double x = sin(dLon) * cos(lat2);
    double y = cos(lat1)*sin(lat2) - sin(lat1)*cos(lat2)*cos(dLon);

    rHead = degrees(atan2(x, y));                      //solve the dynamic theta

    if (rHead < 0)                                     //keep heading error within 360
degrees
        rHead += 360;
}

//=====
=====

void Heading_Error()
{
    error = dHead - rHead;

    if (error > 180)
        error -= 360;

    if (error < -180)
        error += 360;
}

```

```

//=====
=====

void Target_Distance()
{
    float feetlat = fabs(targlat-roverlat) * 364000.0;
    float feetlong = fabs(targlong-roverlong) * 288200.0;

    distance = sqrt(feetlat*feetlat + feetlong*feetlong);
}

//=====
=====

void Travel_Distance()
{
    float travellat = fabs(roverlat-lastlat) * 364000.0;
    float travellong = fabs(roverlong-lastlong) * 288200.0;

    travel = sqrt(travellat*travellat + travellong*travellong);
}

//=====
=====

void Turn(float err)
{
    //Please note that Google Gemini wrote this portion -- this portion was
    added because the rover was recieving how much it should allow a turn,
    however it would
    // turn too much in the old system,
    // causing rover heading to misalign with calculated heading

    // 1. Ignore tiny errors so it doesn't endlessly wiggle
    if (fabs(err) < turnTolerance) return;

    // 2. Calculate exactly how long the motors should spin

```

```

float angle = fabs(err);
uint32_t spinTime = angle * msPerDegree;

uint32_t startSpin = millis();

if (err > 0) {
    // Error is Positive -> Need to turn RIGHT
    // (Flipped HIGH/LOW to account for backwards mounted motors)
    digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
    digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
} else {
    // Error is Negative -> Need to turn LEFT
    // (Flipped HIGH/LOW to account for backwards mounted motors)
    digitalWrite(in1, LOW);  digitalWrite(in2, HIGH);
    digitalWrite(in3, LOW);  digitalWrite(in4, HIGH);
}

// 3. Blast max power to break the dirt friction
analogWrite(enA, 100); //adjust these if slipping on turn
analogWrite(enB, 100);

// 4. Trap the code here until the spin time is complete
while (millis() - startSpin < spinTime) {
    // Do nothing, just spin!
}

// 5. Stop the motors and let the chassis settle
Stop();
delay(500);
}

//=====
=====
=====

void Drive()
{
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(in3, HIGH);

```

```

    digitalWrite(in4, LOW);

    analogWrite(enA, leftMotorspeed);
    analogWrite(enB, rightMotorspeed);
}

//=====
=====

void Stop()
{
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);

    analogWrite(enA, 0);
    analogWrite(enB, 0);
}

//=====
=====

void Drop_Payload()
{
    // Sweep the servo from 0 to 180 degrees
    payloadServo.write(dropAngle);           // Set the servo
position

    payloadDropped = true;
}

```